



SmartCity

Civic Issue Reporting & Resolution Platform

Report it once. Track it live. Hold the city accountable.

A complete study guide to the system: the design approach and the reasoning behind it, the layered architecture, every feature explained with its rules and formulas, and the full database schema.

Team HackInMotion · RICR-HIM-1041

FastAPI · React 19 · PostgreSQL · Leaflet/OpenStreetMap · Claude vision

169 automated tests · 7 migrations · github.com/Nipurnbandi/HackInMotion-RICR-HIM-1041

§ Contents

Part 1 — Overview & Approach

- 1.1 The problem
- 1.2 What the system does
- 1.3 Seven design decisions

Part 2 — Architecture

- 2.1 The six layers
- 2.2 System diagram
- 2.3 Life of a request
- 2.4 Code layout

Part 3 — Feature-wise functionality

- 3.1 Authentication & RBAC
- 3.2 Reporting an issue
- 3.3 Photo upload & validation
- 3.4 Duplicate detection ★
- 3.5 Department routing
- 3.6 Priority scoring ★
- 3.7 SLA escalation ★

- 3.8 Lifecycle & audit trail
- 3.9 Resolution, confirm & reopen
- 3.10 Citizen upvoting
- 3.11 Live city map
- 3.12 Analytics & hotspots ★
- 3.13 Transparency scoring ★
- 3.14 AI photo verification
- 3.15 Notifications
- 3.16 Internationalisation

Part 4 — Database schema

- 4.1 ER diagram
- 4.2 Table-by-table reference
- 4.3 Indexes, constraints, cascades
- 4.4 Enumerations
- 4.5 Migration history

Part 5 — API surface

Part 6 — Revision sheet

How to use this guide. Part 1 gives you the “why” — read it first, because every later rule follows from one of those decisions. Part 3 is the reference body; each feature states its purpose, its exact rules, and the file that implements it. Part 6 condenses every formula and constant onto one page for last-minute revision. Sections marked ★ contain the original algorithms and are the most likely to be asked about.

Part 1 — Overview & Approach

1.1 The problem

Municipal complaint systems fail in four predictable ways. SmartCity is designed around fixing exactly these.

Failure	What actually happens
Duplicate flooding	One pothole generates fifteen tickets. Staff spend their time de-duplicating instead of fixing, and the queue length stops meaning anything.
No feedback loop	The citizen submits into a void — no tracking number, no status, no notification. They assume nothing happened and report again, worsening the first problem.
Manual triage	A human decides which department owns each complaint and which is urgent. This is slow, inconsistent, and the first thing to collapse under load.
No accountability	Nobody — citizen, press, or the city itself — can see which department resolves things and which sits on them.

1.2 What the system does

 Citizen Portal Light theme, phone-first, English + हिन्दी. A five-step report wizard, a live city map to see and upvote what neighbours already reported, and full tracking with the right to confirm or reopen a fix.	 Admin Command Center Dark, data-dense. A priority-ordered work queue holding <i>one row per real-world problem</i> , the same map in dark tiles, live analytics, and resolution tooling with proof photos.	 Public Report Card No login. Every department scored 0–100 and graded A+ to D from its own resolution record, recomputed on every page load.
--	--	--

1.3 Seven design decisions

These are the choices that shape everything else. Understand these and the rest of the system is predictable.

① One case per real-world problem

Reports are not tickets. A report joins a **case**; the case is the unit of work. Fifteen reports of one pothole form one case with fifteen reporters — and the reporter count becomes a *priority signal* rather than queue noise. This single decision is why the admin queue is short, why priority scoring is meaningful, and why resolving once satisfies everyone.

② Configuration in data, not in code

Category → department routing lives in a database table, not an `if/elif` chain. Adding a category or reorganising departments is an `INSERT`, not a redeploy.

③ Per-category constants, not global ones

A pothole and a burst water main are not the same shape of problem, so they don't share a deduplication radius, a persistence window, an urgency, or a deadline. Five independent constants are tuned per category (see §3.4).

④ Security enforced on the server

Every protected route is guarded by a role dependency in FastAPI. The frontend route guard is a convenience for UX; it is never the boundary. A citizen calling an admin URL directly still gets `403`.

⑤ Never block the citizen

Geocoding failure, AI unavailability, or a dead mail server must not prevent a report being filed. Each of those degrades to a fallback — coordinates instead of an address, an `UNVERIFIED` verdict, a queued retry — and the submission still succeeds.

⑥ Append-only history

Status is a single mutable column, but every change to it appends an immutable row to `status_history`. The current state is cheap to query; the full story is never lost.

⑦ Accountability is public by default

The transparency score is unauthenticated and computed live from the same rows the admins work from. There is no separate "public" dataset to massage, and the scoring method is printed on the page.

Part 2 — Architecture

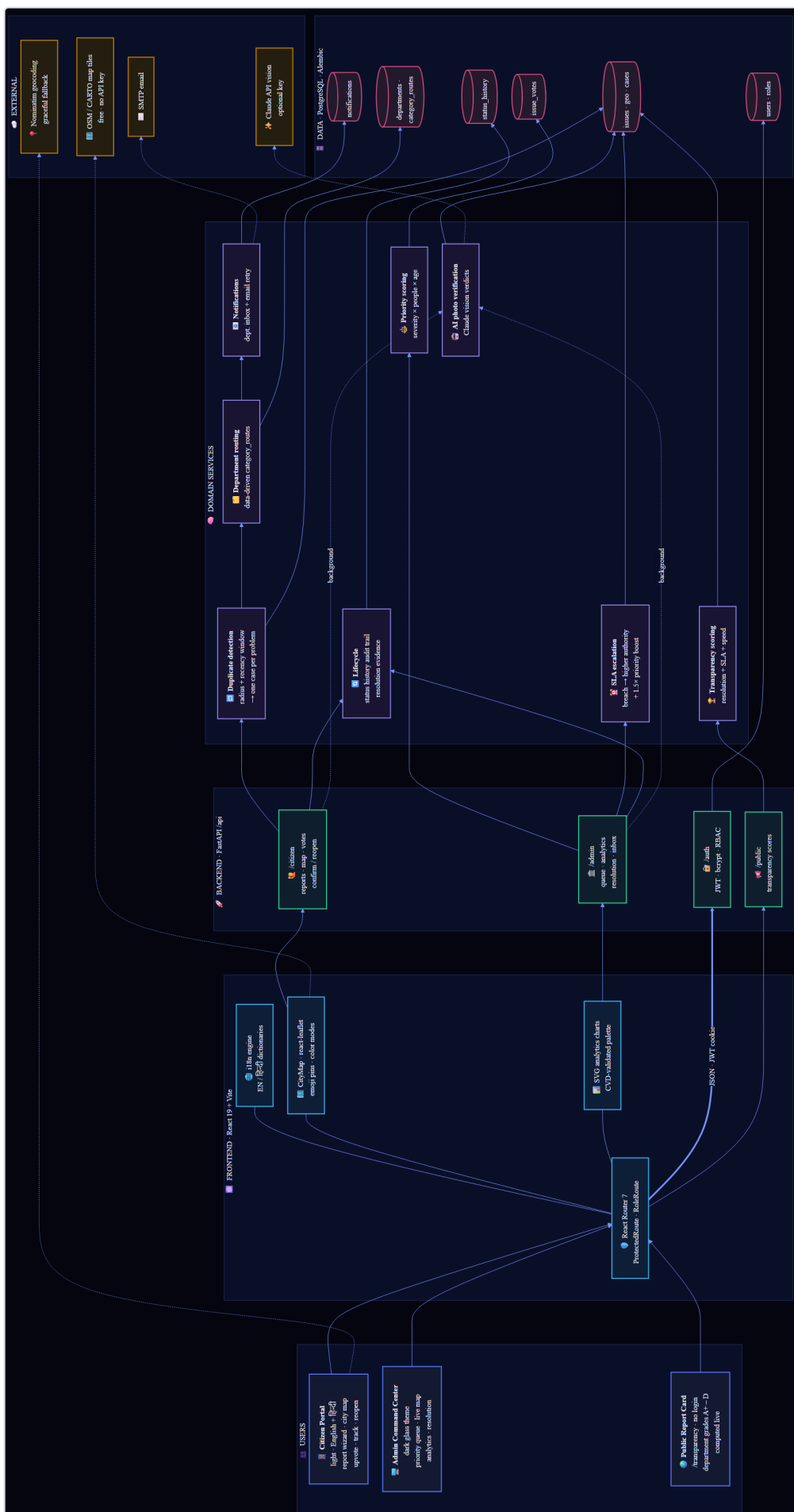
2.1 The six layers

Layer	Technology	Responsibility
Users	Browser (desktop + mobile)	Three distinct surfaces: citizen portal, admin command center, public report card.
Frontend	React 19, Vite, React Router 7	Route guarding, i18n, the Leaflet map component, hand-built SVG charts. Calls the API on a relative <code>/api</code> path.
API	FastAPI, Pydantic v2	Four routers — <code>/auth</code> , <code>/citizen</code> , <code>/admin</code> , <code>/public</code> . Validation, auth, and role enforcement live here; business rules do not.
Domain services	Plain Python	Where the thinking happens: duplicate detection, routing, priority, SLA, lifecycle, transparency, AI verification, notifications. Framework-free and directly unit-testable.
Data	PostgreSQL, SQLAlchemy 2, Alembic	Seven tables, seven migrations. All state, including the audit trail.
External	OSM/CARTO, Nominatim, Claude, SMTP	Map tiles, geocoding, vision verdicts, email. <i>Every one of them is optional or has a fallback.</i>

Why a service layer at all? Routers are thin on purpose. `find_open_case_id`, `run_sla_escalations` and `build_transparency_report` take a database session and plain arguments — no request, no response, no framework. That is what makes 110 backend tests fast and makes the rules readable without tracing HTTP plumbing.

2.2 System diagram

The full architecture diagram is reproduced on the **next page as a full-page plate** — turn the page clockwise to read it. It shows data flow across all six layers; solid lines are direct calls and **dotted lines are background tasks and calls to external services**. Every box corresponds to a real module listed in §2.4.



2.3 Life of a request — submitting a report

#	Layer	What happens
1	Frontend	Wizard validates each step client-side; multipart <code>POST /api/citizen/issues</code> with progress tracking.
2	API	<code>citizen_only</code> dependency decodes the JWT and asserts the role. Pydantic validates the body; the photo is size- and magic-byte-checked.
3	Service	Photo saved under a generated filename. Duplicate detection runs before the insert.
4	Service	Row inserted; <code>tracking_id</code> and <code>case_id</code> assigned; department resolved from the routing table.
5	Data	Creation entry appended to <code>status_history</code> ; a notification row is queued if this is a new case.
6	API	Response returns with the tracking ID — the citizen is done.
7	Background	<i>After</i> the response: email delivery is attempted and the photo is sent for AI verification.

Steps 6 and 7 are the important ordering: nothing slow or external sits between the citizen and their confirmation.

2.4 Code layout

Path	Contents
<code>backend/app/api/</code>	Routers: <code>auth.py</code> , <code>citizen.py</code> , <code>admin.py</code> , <code>public.py</code>
<code>backend/app/services/</code>	Domain logic: <code>citizen</code> , <code>admin</code> , <code>routing</code> , <code>lifecycle</code> , <code>escalation</code> , <code>votes</code> , <code>transparency</code> , <code>city_map</code> , <code>photo_verification</code> , <code>notification</code>
<code>backend/app/core/</code>	<code>config</code> , <code>database</code> , <code>security</code> , <code>dependencies</code> (RBAC), <code>issue_types</code> (the constants), <code>storage</code> , <code>roles</code> , <code>notifier</code>
<code>backend/app/models.py</code>	All seven SQLAlchemy models
<code>backend/alembic/versions/</code>	Seven migrations, <code>001</code> → <code>007</code>
<code>frontend/src/citizen/</code>	Citizen pages, the report wizard, the location picker
<code>frontend/src/admin/</code>	Dashboard, analytics panel, admin service
<code>frontend/src/shared/</code>	<code>CityMap</code> , <code>StatusBadge</code> , <code>i18n.jsx</code> , <code>constants.js</code> (colour palettes)

Part 3 — Feature-wise functionality

3.1 Authentication & role-based access control

Purpose. Two roles with genuinely different interfaces and permissions, enforced where it counts.

Aspect	Behaviour
Password storage	bcrypt via passlib
Token	JWT (HS256) carrying <code>sub</code> , <code>role</code> , <code>type:"access"</code> , <code>exp</code>
Transport	HTTP-only cookie; <code>Authorization: Bearer</code> also accepted
Lifetime	30 minutes by default (<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>)
Enforcement	<code>require_roles(...)</code> FastAPI dependency on every protected route
Privilege escalation	Signup ignores any client-supplied role — always <code>CITIZEN</code> . Admins are made by CLI script or startup bootstrap.

Why another citizen's report returns 404, not 403. A `403` would confirm that the requested tracking ID exists. The lookup therefore treats "doesn't exist" and "isn't yours" identically, so the API cannot be used to enumerate valid IDs. Role mismatches still return `403` — there is nothing to hide about which endpoints are admin-only.

3.2 Reporting an issue

Purpose. Get a usable, located, evidenced report from someone standing on a street with one hand free.

#	Step	Validation
1	Category	One of seven enum values
2	Location	Required; latitude <code>-90...90</code> , longitude <code>-180...180</code> . Address auto-filled, optional
3	Photo	Optional — see §3.3
4	Description	Trimmed, 10–2000 characters
5	Review	Read-only summary; every row links back to its step

Two ways to set the location, neither able to hard-fail: the browser Geolocation API (high accuracy, 10-second timeout) or tapping/dragging the map pin. Either triggers an abortable reverse-geocode; any failure resolves to `null` and the report proceeds on coordinates alone.

On success the server assigns `tracking_id = SMC-{year}-{id:06d}`, a `case_id`, status `SUBMITTED`, and a department.

3.3 Photo upload & validation

Check	Rule	On failure
Non-empty	File has content	400
Size	≤ 5 MB (<code>MAX_UPLOAD_SIZE_BYTES</code>)	413
Declared type	JPEG / PNG / WEBP if a <code>Content-Type</code> is sent	400
Actual bytes	Magic-byte sniff must match JPEG, PNG or WEBP	400

Why sniff the bytes. Extension and `Content-Type` are both attacker-controlled. A `.exe` renamed to `.jpg` with a spoofed header passes every other check and fails this one. Signatures: JPEG `FF D8 FF`, PNG `89 50 4E 47 0D 0A 1A 0A`, WEBP `RIFF ... WEBP` at offset 8.

Storage. Written to `{UPLOAD_DIR}/issues/` or `/resolutions/` under a regenerated name (`uuid4().hex + token_hex(4) + ext`) — the user's filename never touches the filesystem, eliminating path traversal and collisions. The storage layer is a `Protocol`, so an S3 backend is one new class.

3.4 Duplicate detection & case grouping ★

Purpose. Collapse many reports of one problem into a single unit of work, without losing any reporter's ability to track their own submission.

Step 1 — bounding box

The category's radius in metres is converted to degrees, correcting longitude for latitude convergence:

```
lat_delta = radius_m / 111_320
lon_delta = radius_m / max(111_320 * cos(latitude), 1.0)
```

The `cos(latitude)` term matters: a degree of longitude spans ~111 km at the equator but ~96 km at 30° N. Without it the box would be too wide in northern cities. The `max(..., 1.0)` guard prevents division blow-up near the poles.

Step 2 — candidate query

An existing issue matches when **all six** conditions hold:

Condition	Meaning
same <code>category</code>	Same kind of problem
<code>latitude</code> within $\pm$ <code>lat_delta</code>	Inside the box
<code>longitude</code> within $\pm$ <code>lon_delta</code>	Inside the box
<code>created_at</code> > now – window	Recent enough to be the same occurrence
status not <code>RESOLVED/REJECTED</code>	A fixed pothole that returns is a <i>new</i> case
<code>case_id</code> is not null	Already grouped

Ordered `created_at ASC, id ASC`, limit 1 — so the **oldest** case wins and reports always attach to the original, never to a later duplicate.

Step 3 — attach or create

- **Match** → `case_id` = matched case, `is_primary` = False
- **No match** → `case_id` = `CASE-{own id:06d}`, `is_primary` = True

Consequences of the primary / member split

Concern	Behaviour
Admin queue	Lists primaries only — one row per real-world problem
Status	Members have no independent status; they inherit the primary's <i>effective status</i>
Citizen view	Each reporter still sees their own report and tracking ID
Resolution	Resolving the primary satisfies every attached reporter
Reopen	A member reopening reopens the whole case
Counting	<code>citizen_count</code> = distinct citizens; <code>report_count</code> = total reports

Known trade-off — this is a rectangle, not a circle. The corners of the bounding box reach $\approx 1.41\times$ the nominal radius, so a report diagonally 35 m from a pothole can still match a 25 m rule. That is deliberate: it is one indexed range query with no per-row trigonometry (backed by `ix_issues_dup_lookup`), and slight over-matching is the safer error — a wrongly linked report is visible and correctable, while a missed duplicate silently splits a case and re-creates the original problem.

3.5 Department routing

Purpose. Get each report to its owning team with no human triage step.

`resolve_department(db, category)` reads the `category_routes` table (unique on category) and falls back to **General Administration** when no row matches. Seeding is idempotent, so it is safe to re-run on every deploy.

Code	Name	Receives
<code>ROADS</code>	Roads Department	Pothole
<code>ELECTRICAL</code>	Electrical & Streetlighting	Streetlight
<code>SANITATION</code>	Sanitation Department	Overflowing Garbage, Illegal Dumping
<code>WATER</code>	Water & Drainage	Water Leakage, Broken Drainage
<code>PUBLIC_WORKS</code>	Public Works	Damaged Public Property
<code>GENERAL</code>	General Administration	Fallback + all SLA escalations

3.6 Priority scoring ★

Purpose. Order the work queue by real-world impact rather than arrival time.

```
// per case, recomputed on every queue load
people_affected = citizen_count + vote_count
priority        = severity_weight × people_affected × (1 + days_open / 7)
if escalated    → priority × 1.5
```

Factor	Effect
Severity weight (2–5)	A water leak outranks illegal dumping at equal size and age
People affected	Distinct reporters plus upvotes — mass-affecting problems climb
Age factor	+1× per week waited; nothing can be ignored indefinitely
Escalation boost	A breached case outranks comparable un-breached ones

Worked example — a 3-day-old water leak (weight 5), 2 reporters, 1 upvote:

```
people_affected = 2 + 1 = 3
priority = 5 × 3 × (1 + 3/7) = 5 × 3 × 1.4286 = 21.4
after an SLA breach: 21.4 × 1.5 = 32.1
```

The UI colours the badge **≥20 high**, **≥10 medium**, else low.

3.7 SLA escalation ★

Purpose. Guarantee that an ignored case eventually forces itself onto someone's desk.

The sweep runs opportunistically on *every* admin dashboard, analytics and queue request — there is no cron job or worker process to keep alive.

A case escalates when all four hold: it is a primary; its status is not closed; `escalated_at IS NULL` ; and `now ≥ created_at + SLA_DAYS[category]` .

Four actions follow:

1. `escalated_at = now` — the idempotency guard, so a case escalates **exactly once** no matter how many times an admin reloads.
2. A history entry naming the breached limit, visible in the reporter's own trail.
3. A notification to General Administration beginning `SLA BREACH` .
4. The ⚠ badge and the ×1.5 priority multiplier.

Why opportunistic and not scheduled. A background scheduler is another moving part that can silently die, and on free hosting tiers it may not run at all. Tying the sweep to admin reads means escalation is guaranteed to have happened before anyone *looks* at the queue — which is the only moment it actually matters. `escalated_at` makes repeated runs free.

3.8 Lifecycle & audit trail

```
SUBMITTED → UNDER_REVIEW → IN_PROGRESS → RESOLVED
      ↓           ↓           ↓
      └──────────┘ → REJECTED

RESOLVED —(citizen reopens)—→ UNDER_REVIEW
```

Status	Shown to the citizen	Set by
SUBMITTED	"We have received your report."	System
UNDER_REVIEW	"A city officer is verifying the details."	Admin / citizen reopen
IN_PROGRESS	"Work has started on this issue."	Admin
RESOLVED	"The issue has been fixed."	Admin
REJECTED	Could not be actioned	Admin

Why transitions aren't restricted. Real municipal work doesn't advance one step at a time — a crew may fix something the same morning it is reported. Rather than forcing a fictional sequence, the system allows any transition and guarantees instead that *every* transition is recorded. The audit trail, not the state machine, is the control.

Every event appends an immutable row to `status_history` : old status, new status, note, optional photo, and whether a `CITIZEN` or `ADMIN` caused it. Recorded events are submission, admin status change, resolution note, proof upload, SLA escalation, citizen confirmation and citizen reopen. Migration `005` backfilled a synthetic creation entry for pre-existing issues so no report has an empty history.

Case-level assembly: a duplicate reporter sees their own entries *plus* the shared case's entries, merged chronologically — never another reporter's private rows.

3.9 Resolution evidence, confirm & reopen

Admin side. A resolution note (≤ 2000 chars) and a proof-of-resolution photo are two independent operations, so a note can be attached while work is still in progress. The photo runs the identical validation as citizen uploads and triggers AI verification.

Citizen side. Once the case is `RESOLVED` the reporter is asked whether it actually looks fixed:

Action	Status effect	Recorded as
Confirm	Unchanged (stays <code>RESOLVED</code>)	"Reporter confirmed the problem is fixed."
Reopen	Back to <code>UNDER_REVIEW</code>	"Reporter reopened the report — the problem is not fixed."

Both act on the **case primary**. If any one of five reporters reopens, the case returns to the queue for everyone — a fix that didn't work cannot be closed by satisfying one row. Attempting either on a non-resolved case returns `400`.

Why confirmation isn't a sixth status. Adding `VERIFIED` would double the states the UI, filters and analytics must handle, to record a fact that is inherently historical. It lives in the audit trail instead — the state machine stays small.

3.10 Citizen upvoting

Purpose. Let people amplify problems they didn't report, and feed that into prioritisation.

Rule	Behaviour
Target	Must be a primary issue; otherwise 404
Self-vote	Blocked if the citizen has any report in that case — 400
Toggle	Voting again withdraws the vote — it is a switch, not an increment
Uniqueness	UNIQUE(issue_id, citizen_id) — double-submission cannot double-count

3.11 Live city map

One component serves both portals, differing only by a `theme` prop.

Aspect	Behaviour
Data	Primary issues with coordinates only — members folded into their case
Tiles	OpenStreetMap (citizen, light) / CARTO Dark Matter (admin, dark)
Markers	<code>L.divIcon</code> emoji chips inside a coloured ring
Colour modes	By status or by category , toggled in the toolbar
Viewport	Auto <code>fitBounds</code> over all pins, capped at zoom 16
Legend	Built from what is actually on screen, with live counts

Accessibility. Both palettes were validated for contrast and colour-vision deficiency against the dark surface, and *state never rides on hue alone*: resolved pins are dimmed and scaled down, rejected pins are hollow with a dashed ring. Status stays readable in greyscale.

3.12 Analytics & hotspot detection ★

Every figure is aggregated per request — nothing cached, precomputed or seeded.

Metric	Definition
<code>total_issues</code>	Count of primary issues = real-world problems
<code>total_reports</code>	Count of all issue rows = submissions
<code>open_issues</code>	Primaries not RESOLVED/REJECTED
<code>avg_resolution_days</code>	Mean <code>updated_at</code> - <code>created_at</code> over resolved primaries; <code>null</code> if none
<code>by_category</code> / <code>by_status</code>	Counts including zeros , so chart axes stay stable
<code>departments</code>	Per-department totals, open, resolved, average; “Unassigned” row only if needed

The gap between `total_reports` and `total_issues` is the direct measure of how much noise duplicate detection removed.

Hotspot algorithm

1. **Grid.** Round latitude/longitude to **3 decimal places** → ~110 m × 110 m cells.

- 2. **Weight by reports, not cases.** One case with six reporters counts as six, correctly outranking three unrelated single-report cases.
- 3. **Threshold.** Cells with fewer than **2 reports** are discarded.
- 4. **Label.** Most common category; address from the *most recent* issue that has one (older addresses may be stale).
- 5. **Rank & cap.** By report count then case count; top **8** returned.

3.13 Public transparency scoring ★

Purpose. Make departmental performance externally visible and impossible to quietly massage.

```
score = 100 × ( 0.50 × resolution_rate
               + 0.30 × on_time_rate
               + 0.20 × speed_factor )

resolution_rate = resolved ÷ total
on_time_rate    = resolved within category SLA ÷ resolved
speed_factor    = max(0, 1 - avg_resolution_days / 14)
```

Score	Grade
≥ 85	A+
≥ 70	A
≥ 55	B
≥ 40	C
< 40	D

Two deliberate guards. `on_time_rate` and `speed_factor` are **0** when nothing is resolved — a department that never closes anything scores 0 rather than being flattered by an empty average. A department with **zero cases** gets `null`, rendered “No data”, never an F.

Worked example — one pothole case resolved in 2 days (SLA 7):

```
resolution_rate = 1/1    = 1.0
on_time_rate    = 1/1    = 1.0      (2 ≤ 7)
speed_factor    = 1 - 2/14 = 0.857
score = 100 × (0.5 + 0.3 + 0.2×0.857) = 97 → A+
```

3.14 AI photo verification

Purpose. Give the reviewing officer a cheap first-pass signal on whether a photo matches its claim.

Photo	Question asked
Report photo	“Does this plausibly show a {category}?” — given the description, explicitly tolerant of casual phone framing and lighting
Proof photo	“Does this plausibly show that issue repaired or resolved?”

Verdicts are `MATCH`, `MISMATCH` or `UNVERIFIED`, obtained via a JSON-schema structured output (`{match, reason}`) at low effort, with the reason truncated to 500 characters and shown as a badge tooltip.

Failure is total and silent — by design. Missing API key, model refusal, network error or malformed response all resolve to `UNVERIFIED` with an explanatory note; nothing raises. Both checks run as **background tasks**, so the HTTP response returns before the model is even called. A report can never be blocked, delayed or rejected by the AI — the verdict is advisory information for a human, not a gate.

3.15 Department notifications

Two-phase, so a broken mail server can never fail a citizen's submission.

- **Phase 1 (synchronous).** A notification row is written when a new **primary** case is created. Duplicates do not re-notify.
- **Phase 2 (background).** Every row with `sent_at IS NULL` is attempted; on success the timestamp is stamped, on failure the row stays pending and the *next* submission retries it. Delivery is at-least-once and nothing is lost.

`NOTIFIER=console` logs the email (safe default); `email` sends via SMTP. The admin inbox shows the 50 most recent with an unread count and distinguishes delivered from pending.

3.16 Internationalisation

A dependency-free layer: `t(key, fallback)` with English fallbacks inline in the components.

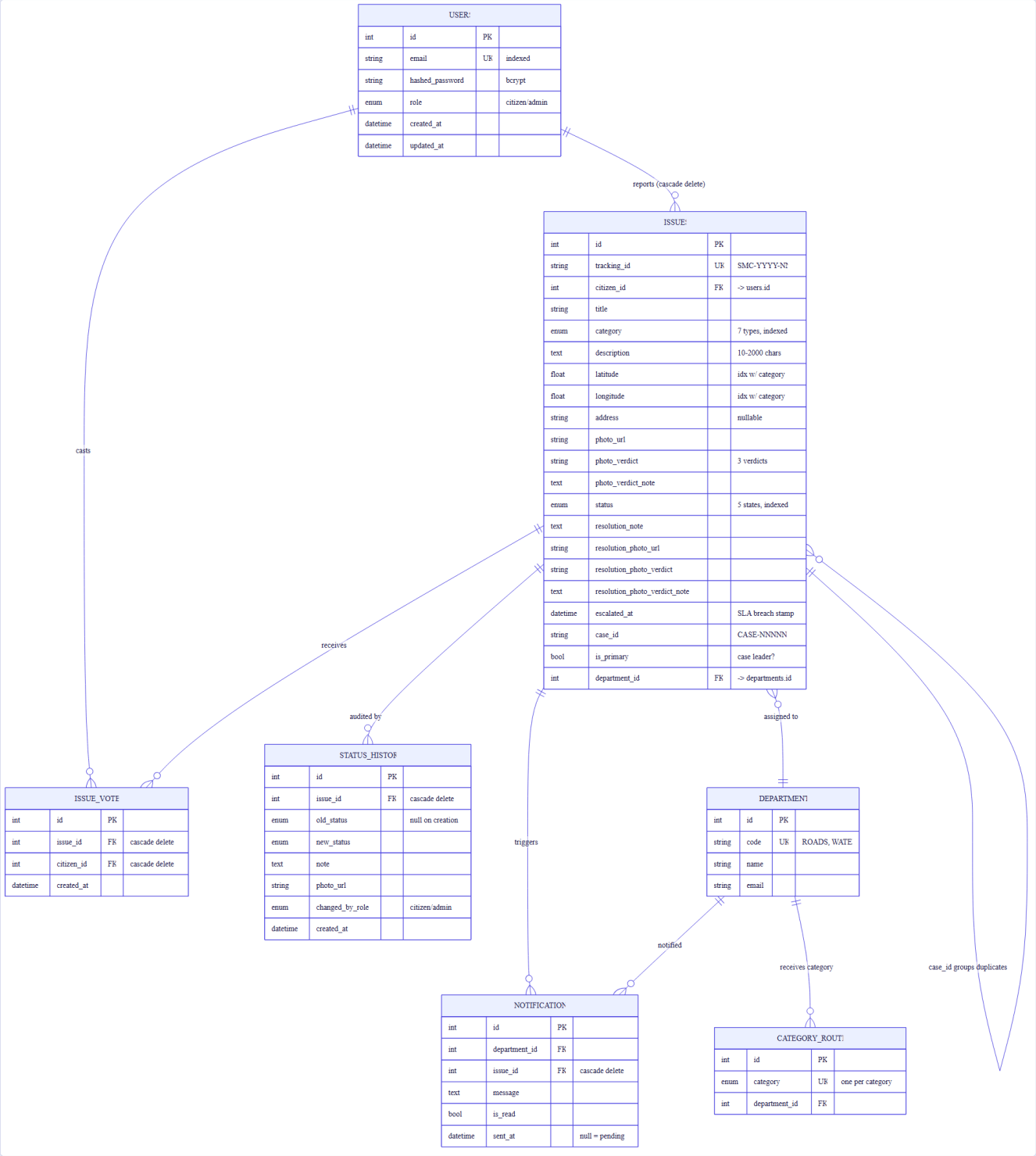
Why fallbacks are inline. A missing translation key returns the English string that sits right there in the JSX — so a partial translation can never render a blank space or a raw `key.name`. Adding a third language is one dictionary object and zero component changes.

Covered: navigation, dashboard, the full report wizard, categories and hints, statuses and descriptions, the timeline, My Reports, issue details, activity history, confirm/reopen prompts, the map toolbar/legend/popups, and AI verification notes. Persisted in `localStorage`; unreadable storage falls back to English without error.

Part 4 — Database schema

PostgreSQL, seven tables, accessed through SQLAlchemy 2 and evolved with seven Alembic migrations. Every table carries `created_at` and `updated_at` from a shared `TimestampMixin`.

4.1 Entity–relationship diagram



All seven tables with their columns, keys and relationships. Note the self-relationship on `issues`: `case_id` groups duplicate reports into one case. Column-level detail follows in §4.2.

4.2 Table-by-table reference

users

Column	Type	Notes
id	Integer	PK, autoincrement
email	String(255)	Unique , indexed — the login identifier
hashed_password	String(255)	bcrypt hash; the plaintext is never stored
role	Enum	CITIZEN (default) or ADMIN ; non-native enum → stored as VARCHAR
created_at / updated_at	DateTime(tz)	Server defaults; updated_at auto-updates

issues — the central table (reports *and* cases)

Column	Type	Notes
id	Integer	PK
tracking_id	String(32)	Unique , indexed. SMC -{year}-{id:06d}
citizen_id	Integer FK	→ users.id , ON DELETE CASCADE , indexed
title	String(255)	Derived from the category label
category	Enum	7 values, indexed ; drives radius, window, severity, SLA, routing
description	Text	10–2000 characters (validated at the schema layer)
latitude / longitude	Float, nullable	Part of the composite dedup index
address	String(500), nullable	Reverse-geocoded; null when lookup failed
photo_url	String(500), nullable	Public URL of the stored evidence photo
photo_verdict	String(20), nullable	MATCH / MISMATCH / UNVERIFIED
photo_verdict_note	Text	One-sentence AI reason (≤500 chars)
status	Enum	5 values, indexed . Members inherit the primary's effective status
resolution_note	Text	Admin's description of the fix; visible to the reporter
resolution_photo_url	String(500), nullable	Proof-of-resolution image
resolution_photo_verdict	String(20), nullable	AI verdict on the proof photo
resolution_photo_verdict_note	Text	AI reason for the proof verdict
escalated_at	DateTime(tz), nullable	SLA idempotency guard — non-null means already escalated
case_id	String(32), nullable	Indexed . CASE -{id:06d} — the duplicate-grouping key

Column	Type	Notes
is_primary	Boolean	Default true. Primaries appear in the admin queue and on the map
department_id	Integer FK, nullable	→ departments.id, indexed

status_history — append-only audit trail

Column	Type	Notes
issue_id	Integer FK	→ issues.id, CASCADE , indexed
old_status	Enum, nullable	Null on the creation entry
new_status	Enum	Equals old_status for note-only / photo-only events
note	Text	Human-readable description of the event
photo_url	String(500), nullable	Set when the event carried a photo
changed_by_role	Enum	CITIZEN → “Reporter”, ADMIN → “City administration”

issue_votes

Column	Type	Notes
issue_id	Integer FK	→ issues.id, CASCADE, indexed
citizen_id	Integer FK	→ users.id, CASCADE, indexed
UNIQUE(issue_id, citizen_id) — uq_issue_votes_issue_citizen. Enforces one vote per citizen per case at the database level, so a double-submitted request cannot double-count.		

departments & category_routes

Table	Column	Notes
departments	code	Unique , indexed — ROADS, WATER, ...
	name	Display name
	email	Notification recipient
category_routes	category	Unique — at most one route per category
	department_id	FK → departments.id

notifications

Column	Type	Notes
department_id	Integer FK	→ departments.id, indexed
issue_id	Integer FK	→ issues.id, CASCADE
message	Text	Full email body
is_read	Boolean	Inbox read state, default false
sent_at	DateTime(tz), nullable	Null = pending — the retry queue marker

4.3 Indexes, constraints & cascades

Object	Purpose
ix_issues_dup_lookup (category, latitude, longitude)	The performance-critical one. Backs the duplicate-detection range query so it never scans the table
ix_issues_case_id	Case grouping — assembling all reports of a case
ix_issues_status, ix_issues_category	Queue filters and analytics aggregation
ix_issues_department_id	Department-filtered queue
uq_issue_votes_issue_citizen	One vote per citizen per case
Unique: users.email, issues.tracking_id, departments.code, category_routes.category	Identity and routing integrity

Cascade behaviour. Deleting a user deletes their issues (ORM `delete-orphan` plus a database-level `ON DELETE CASCADE`). Deleting an issue deletes its history rows, votes and notifications. Departments are never cascade-deleted — they are reference data.

4.4 Enumerations

All enums use `native_enum=False` , so they are stored as VARCHAR with a check constraint rather than a PostgreSQL `TYPE` .

Why avoid native enums. Altering a native PostgreSQL enum requires `ALTER TYPE` , which is awkward inside a transactional migration and cannot easily remove a value. Storing strings keeps every migration a plain, reversible column operation — worth far more during rapid iteration than the few bytes a native enum saves.

Enum	Values
role	CITIZEN, ADMIN
issue_status	SUBMITTED, UNDER_REVIEW, IN_PROGRESS, RESOLVED, REJECTED
issue_category	STREETLIGHT, POTHOLE, GARBAGE_OVERFLOW, WATER_LEAKAGE, DAMAGED_PUBLIC_PROPERTY, ILLEGAL_DUMPING, BROKEN_DRAINAGE

4.5 Migration history

Revision	What it added
001	Initial <code>users</code> and <code>issues</code>
002	Civic issue fields — category, geolocation, photo, tracking ID
003	Case grouping — <code>case_id</code> , <code>is_primary</code> , the dedup index
004	Departments, category routes, notifications — <i>seeds the six departments and seven routes</i>
005	Status history + resolution evidence — <i>backfills a creation entry for every existing issue</i>
006	<code>escalated_at</code> and the <code>issue_votes</code> table
007	AI photo-verification verdict columns

Part 5 — API surface

Base path `/api` . JWT via HTTP-only cookie or bearer header. Citizen routes reject admins and vice versa.

Authentication

Method	Path	Purpose
POST	<code>/auth/signup</code>	Create a citizen account (role escalation blocked)
POST	<code>/auth/login</code>	Authenticate; sets the auth cookie
POST	<code>/auth/logout</code>	Clear the cookie
GET	<code>/auth/me</code>	Current user identity and role

Citizen

Method	Path	Purpose
GET	<code>/citizen/dashboard</code>	Greeting, per-status stats, recent reports
GET	<code>/citizen/map</code>	All active cases for the live map (with vote state)
GET	<code>/citizen/issues</code>	Own reports — <code>status</code> , <code>search</code> , <code>page</code> , <code>page_size</code>
GET	<code>/citizen/issues/stats</code>	Own per-status counts
POST	<code>/citizen/issues</code>	Submit a report (multipart) — runs dedup + routing
GET	<code>/citizen/issues/{id}</code>	One report with full status history
POST	<code>/citizen/issues/{id}/confirm</code>	Confirm the fix
POST	<code>/citizen/issues/{id}/reopen</code>	Reopen the whole case
POST	<code>/citizen/issues/{id}/vote</code>	Toggle an upvote

Admin

Method	Path	Purpose
GET	/admin/dashboard	Headline totals runs SLA sweep
GET	/admin/analytics	All metrics + hotspots runs SLA sweep
GET	/admin/map	All active cases for the dark map
GET	/admin/departments	Departments with open-case counts
GET	/admin/issues	Work queue , priority-sorted runs SLA sweep
PUT	/admin/issues/{id}	Update status / title / description / resolution note
POST	/admin/issues/{id}/resolution-photo	Upload proof of resolution
DELETE	/admin/issues/{id}	Remove a report
GET	/admin/notifications	Inbox with unread count and delivery state
POST	/admin/notifications/{id}/read	Mark as read

Public — no authentication

Method	Path	Purpose
GET	/public/transparency	Live department report card with methodology

Part 6 — Revision sheet

Everything worth memorising, on one spread.

The master constants table

Five independent behavioural constants per category — the heart of the system.

Category	Dedup radius	Dedup window	Severity	SLA	Department
Pothole	25 m	60 d	4	7 d	Roads
Streetlight	30 m	45 d	3	7 d	Electrical
Damaged Public Property	30 m	60 d	3	10 d	Public Works
Overflowing Garbage	50 m	3 d	2	2 d	Sanitation
Illegal Dumping	60 m	7 d	2	4 d	Sanitation
Water Leakage	80 m	10 d	5	3 d	Water & Drainage
Broken Drainage	80 m	30 d	5	5 d	Water & Drainage

Read across a row: a burst main is detected over a wide area (80 m), assumed to persist only 10 days, is maximally severe (5), and must be fixed in 3 days. A pothole is the opposite profile on every axis except severity.

Every formula

```
- duplicate detection -
lat_delta = radius_m / 111_320
lon_delta = radius_m / max(111_320 * cos(latitude), 1.0)

- priority score (admin queue order) -
priority = severity * (reporters + votes) * (1 + days_open / 7)
          * 1.5 if SLA-breached

- transparency score (public report card) -
score = 100 * (0.50*resolution_rate + 0.30*xon_time_rate + 0.20*speed_factor)
speed_factor = max(0, 1 - avg_resolution_days / 14)
grades: 85+ A+ · 70+ A · 55+ B · 40+ C · else D

- hotspots -
cell = round(lat, 3), round(lon, 3)           ≈110 m grid
keep cells with ≥ 2 reports, rank by reports, take top 8
```


Numbers to remember

Value	Meaning
10 – 2000	Description length, characters
5 MB	Maximum photo size; JPEG / PNG / WEBP only, verified by magic bytes
30 minutes	Default JWT lifetime
111,320	Metres per degree of latitude — the dedup conversion constant
×1.5	Priority multiplier on an SLA breach
14 days	Speed-score baseline in the transparency formula
3 dp / 2 / 8	Hotspot grid precision, minimum reports, maximum returned
50 / 30 / 20	Transparency weights (%): resolution, on-time, speed
7 / 7 / 169	Tables, migrations, automated tests

Likely exam questions

Question	One-line answer
How do you stop duplicate reports?	Per-category radius + time window + open-status check groups them into one case; the reporter count then <i>raises</i> priority.
Why is the dedup box rectangular?	One indexed range query, no per-row trigonometry; over-matching is the safer failure.
How is routing extensible?	It's a database table, not code — a new category is an INSERT .
What stops an escalation firing repeatedly?	The escalated_at timestamp — the sweep only selects rows where it is null.
Where is authorisation enforced?	Server-side FastAPI dependencies; the frontend guard is UX only.
What if the AI or the mail server is down?	Nothing fails — UNVERIFIED verdict, or a pending row retried on the next submission.
Can a department game the public score?	No — it is computed live from the same rows they work, and rejecting instead of resolving lowers the resolution rate.
Who decides a case is really fixed?	The reporter — confirm, or reopen the entire case for everyone.